

# The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch17\_vision

AYuSh

# Contents

|                                  |   |
|----------------------------------|---|
| Chapter 17: Computer Vision..... | 3 |
|----------------------------------|---|

# Chapter 17: Computer Vision

---

Chapter 14 built the convolutional toolkit and Chapter 16 the transformer; this chapter is where they earn a living. Vision interviews rarely stop at "explain a CNN" — they ask what happens when the answer is not one label per image: find every object and box it (detection), label every pixel (segmentation), or connect images to language (CLIP). Each task forces an architectural idea worth deriving: detection needs a way to propose *where* to look and a metric that scores localization (IoU, mAP, NMS — the trio every candidate should be able to code cold); segmentation needs to recover the spatial resolution that classification networks deliberately throw away (U-Net's skip connections); and the Vision Transformer needs a way to turn a grid of pixels into a set of tokens (patches), after which Chapter 16 applies verbatim.

The chapter's experiments: IoU verified on hand-computable cases and NMS collapsing 13 noisy detections to 3 (Listing 1); average precision computed by hand on a seven-detection example where a duplicate and a hallucination each cost exactly what the rules say they cost, and AP@0.5/0.75/0.9 falling  $0.886 \rightarrow 0.371 \rightarrow 0.200$  as the localization bar rises (Listing 2); anchor assignment and the Faster R-CNN box encoding with an exact round-trip (Listing 3); a YOLO-style grid detector trained from scratch to 0.81 recall on synthetic scenes (Listing 4); a real miniature U-Net whose skip connections cut boundary-pixel errors from 5.9% to 2.4% (Listing 5); a from-scratch ViT — patch embedding, CLS token, two transformer blocks, gradient-checked end to end at  $1.6 \times 10^{-10}$  — reaching 92.4% on digits, with its CLS attention maps rendered (Listing 6); a two-tower CLIP trained with symmetric InfoNCE where making the temperature learnable is worth 3 points and the scale converges to 17 on its own (Listing 7); and an augmentation study in which a 1-pixel test-time jitter collapses an unaugmented model from 0.962 to 0.343 while the shift-augmented one holds 0.872 — and horizontal flip, a *false* invariance for digits, buys nothing and costs clean accuracy (Listing 8).

## Classification, backbones, and transfer

Image classification is Chapter 14's material: convolutional backbones (ResNet-family) or ViTs (below) pretrained on large corpora, fine-tuned per task — the freeze-vs-finetune trade measured in Chapter 14's Listing 7 applies unchanged. What matters here is the backbone's role as *currency*: detection and segmentation architectures are mostly answers to the question "how do we bolt localization onto a classification backbone without retraining vision from scratch?" A backbone's feature pyramid — high-resolution/low-semantics early layers, low-resolution/high-semantics late layers — is the raw material; FPN (Feature Pyramid Network) fuses top-down semantics with lateral high-resolution detail so that objects of every scale get features worth detecting from, and some flavor of it sits inside nearly every modern detector.

## The detection metrics: IoU, NMS, mAP

**Intersection-over-Union** is the geometry of "did you find it": intersection area over union area of predicted and ground-truth boxes. Listing 1's hand cases calibrate intuition — identical boxes 1.0, half-overlapping unit squares 1/3, corner-touching quarter overlap 1/7, disjoint 0 — and the practical thresholds follow: 0.5 is the classic "found it," 0.75 is tight, and COCO averages over 0.50:0.95 to reward precise boxes. Detectors emit *many* boxes per object (that redundancy is by design — something must fire everywhere an object might be), so **non-maximum suppression** cleans up: sort by confidence, greedily keep the best box, delete everything overlapping it above a threshold, repeat. Listing 1 runs it on 13 noisy detections around two objects: 3 survive (the figure shows before/after). Its failure mode is the standard follow-up: two genuinely overlapping objects (crowds) get merged — hence Soft-NMS (decay scores instead of deleting) and DETR's set prediction (below), which needs no NMS at all.

**mAP** is where interviews separate metric-quoters from metric-understanders, and Listing 2 computes it with no library: sort all detections by confidence; walk down the list matching each to the best unclaimed ground-truth box of its class (IoU above threshold; *each GT claimable once* — a duplicate detection of an already-claimed object is a false positive, which is exactly how mAP punishes NMS failures); accumulate precision and recall at each rank; take the precision *envelope* (monotone interpolation) and integrate it over recall. The listing's seven detections include one duplicate and one hallucination, and the printed precision/recall arrays show each doing its damage — AP 0.886. Mean over classes gives mAP; mean over IoU thresholds 0.50:0.95 gives COCO mAP, and the listing's threshold sweep (0.886/0.371/0.200 at 0.5/0.75/0.9) shows how hard the tight-localization criterion is. The PR-curve figure makes the envelope construction visible.

## Two-stage detectors: the R-CNN family

The R-CNN line is an efficiency ratchet, and telling it as one is the clean interview answer. **R-CNN** (2014): ~2000 external region proposals (selective search), each warped and pushed through the CNN *separately* — thousands of forward passes per image. **Fast R-CNN**: run the backbone *once*, then crop each proposal's features from the shared feature map with **RoI pooling** (fixed-size grid over the region, max within cells) — the redundant convolution gone, but proposals still external. **Faster R-CNN**: replace selective search with the **Region Proposal Network** — a small conv head sliding over the feature map, scoring objectness and regressing box offsets at every position for a set of reference **anchors** — making the whole pipeline one trainable network: RPN proposes, RoI head classifies and refines. **Mask R-CNN** adds a per-RoI mask branch (below) and fixes RoI pooling's coordinate quantization with **RoIAlign** (bilinear sampling — the quantization error that pooling introduces is irrelevant for classification but fatal for pixel-accurate masks).

Anchors deserve their own listing because every anchor-based detector shares the machinery. Listing 3 builds a  $4 \times 4 \times 3$  anchor grid, assigns positives (best anchor per GT, plus any anchor above IoU 0.5) and negatives (below 0.3, with the 0.3–0.5 band *ignored* — genuinely ambiguous anchors would only inject label noise), and implements the Faster R-CNN box parameterization: center offsets normalized by anchor size, width/height as *log*-ratios — scale-invariant targets, all  $O(1)$ , which is what makes box regression learnable — with an exact encode→decode round-trip (error 0.0). The figure shows the grid, the two GT boxes, and which anchors went positive.

## One-stage detectors: YOLO, SSD, and the class-imbalance tax

One-stage detectors skip proposals: predict boxes *densely*, directly from the feature map. **YOLO** divides the image into an  $S \times S$  grid; the cell containing an object's center owns it and predicts confidence, offsets, and size — one pass, real-time. **SSD** densifies further with multi-scale anchor predictions from several feature-map resolutions. The structural cost: a dense detector scores tens of thousands of locations of which a handful are objects — an extreme class imbalance that swamps the loss with easy negatives. That is precisely Chapter 12's focal-loss story: **RetinaNet**'s  $(1 - p_t)^{\gamma}$  down-weighting of easy examples (measured there at up to  $4600\times$  hard:easy ratio) is what made one-stage accuracy match two-stage. Listing 4 builds the YOLO core from scratch — a  $3 \times 3$  grid over synthetic scenes, per-cell (objectness, center, size) with sigmoid parameterization, responsible-cell loss — and trains it to recall 0.81 / precision 0.83 / mean matched IoU 0.68; the figure shows its boxes on unseen images. Every real detector is this listing plus depth, anchors, multi-scale, and better loss weighting.

**DETR** closes the design arc: a transformer decoder with  $N$  learned *object queries* cross-attends (Chapter 16) into backbone features and emits a *set* of box+class predictions, trained with Hungarian (bipartite) matching so each GT is assigned to exactly one query. No anchors, no NMS — the matching does the deduplication that NMS approximated. Costs to volunteer: slow convergence (fixed by deformable attention variants) and small-object weakness in the original.

## Segmentation: semantic, instance, panoptic

Three problems, one axis of "what counts as a thing." **Semantic** segmentation labels every pixel by class — two dogs merge into one dog-region. **Instance** segmentation separates them — mask per object, background unlabeled. **Panoptic** unifies: every pixel gets a class *and* things get instance identities (stuff like sky/road gets class only). Architecturally: semantic = dense per-pixel classification (FCN lineage); instance = detection plus a mask head (Mask R-CNN predicts a small binary mask *inside each RoI* — per-class masks, sigmoid per pixel, so classes don't compete for mask shape); panoptic = both branches fused.

The semantic workhorse is the **U-Net**: an encoder that downsamples to build context, a decoder that upsamples back to full resolution, and — the actual idea — **skip connections** copying each encoder resolution's features across to the same-resolution decoder stage. The deep path knows *what*; only the early, high-resolution features still know precisely *where*; concatenation gives the decoder both. Listing 5 builds a real one — 3×3 convs via im2col with hand-written backward, mean-pool down, nearest-neighbor up, skip concatenation — and ablates the skip on noisy-disk segmentation: Dice 0.983 → 0.993, and on *boundary pixels specifically* (where localization lives) accuracy 0.941 → 0.976, a 2.5× reduction in boundary errors. The figure shows the no-skip mask's rounded, uncertain edges against the skip version. Loss notes for interviews: per-pixel cross-entropy is standard; Dice loss (optimize the overlap metric directly) fights foreground-background imbalance; boundary losses sharpen edges.

## Vision Transformers

ViT (Dosovitskiy et al., 2020) is Chapter 16 applied to pixels with one new idea: **patches are tokens**. Cut the image into  $16 \times 16$  patches, flatten each, project linearly to  $d$  — that projection is the entire "visual front end" — prepend a learnable CLS token, add positional embeddings (learned; position is 2-D but a 1-D learned table works), and run a standard *encoder* (no causal mask — an image has no autoregressive order; bidirectional attention is exactly right, the BERT configuration, not the GPT one). Classify from the CLS token's final state. Listing 6 builds all of it from scratch on 8×8 digits — 2×2 patches giving 16 tokens, two pre-norm blocks of 4-head attention, the complete backward through patch embedding, positional table, CLS, attention, FFN, and head, gradient-checked at  $1.6 \times 10^{-10}$  — and reaches 92.4% in three epochs. Note it *loses* to Chapter 12's plain MLP (96.0%) on the same data: with 1,400 training images the transformer's weak inductive bias is a liability, which is the honest miniature of ViT's headline finding — CNNs win small data on their baked-in locality and translation equivariance; ViT wins once pretraining data is large enough that attention can *learn* those priors and then some. The figure renders CLS-token attention over the patch grid: the model discovers where the ink is. Two follow-ups worth having ready: attention cost is quadratic in patch count, so higher resolution means smaller-patch/windowed variants (Swin's shifted windows restore a convolution-like locality hierarchy); and DeiT showed distillation from a CNN teacher substitutes for much of the data appetite.

## CLIP: vision meets language

CLIP (Radford et al., 2021) trains an image tower and a text tower to agree: embed both into one space, and on a batch of  $B$  matched pairs push each image toward *its* caption and away from the other  $B - 1$ , symmetrically — the InfoNCE contrastive loss over the in-batch similarity matrix, with a *learnable temperature* scaling the logits. Zero-shot classification falls out: embed the class names ("a photo of a dog"), embed the image, pick the nearest caption — classification as retrieval, no task-specific training. Listing 7

builds the miniature: linear image tower, embedding-table text tower, L2 normalization onto the unit sphere (with the normalization's backward — the tangent-space projection — done by hand), symmetric cross-entropy, and nearest-caption classification at 0.937. The two measured lessons: the learnable temperature is worth 3 points over fixed scale 1.0 and converges to  $\sim 17$  on its own (CLIP's converged  $\sim 100$  — unit-sphere cosines live in  $[-1, 1]$ , so without a large scale the softmax cannot commit); and the listing flags in code why in-batch negatives are subtle — two images of the same class in one batch make each other's captions *false negatives*, which real CLIP absorbs with enormous, diverse batches (32,768 in the paper; batch size is a first-class hyperparameter for contrastive learning, not a memory detail). The similarity-matrix figure shows the diagonal the loss carved. CLIP's embedding space is what powers Chapter 24's multimodal retrieval and the text encoders of diffusion models (Chapter 18).

## Augmentation: invariance you assert, not data you get

Augmentation is usually filed under "more data for free," and Listing 8 is built to correct that: an augmentation is a *claim that the label is invariant to a transform*, and false claims are paid for. On 300 training digits: the unaugmented model scores 0.962 clean but 0.343 under a mere  $\pm 1$ -pixel test-time jitter — it never learned translation tolerance because centered training data never asked for it. Shift augmentation (zero-pad shifts, mixed 50/50 with originals — both details matter; Chapter 13 found wrap-around `np.roll` shifts actively harmful) trades 1 clean point for jitter robustness  $0.343 \rightarrow 0.872$ . Horizontal flip — a *true* invariance for natural photos, a *false* one for digits and text — costs clean accuracy and buys zero robustness; the figure of flipped digits makes the label damage visible. The standard menu (crops, flips, color jitter, Mixup/CutMix, RandAugment) is therefore domain knowledge, not a default: medical images have canonical orientations, OCR must never flip, and aggressive Mixup on small fine-grained datasets can erase the distinctions being learned. Test-time augmentation (average predictions over transforms) is the inference-side cousin — a small accuracy rental paid for with multiplied compute.

## Code implementations

### Listing 1 — IoU and non-maximum suppression from scratch

```
"""Listing 1: IoU and non-maximum suppression from scratch."""
import numpy as np
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.patches as mp
rng = np.random.default_rng(0)

def iou(a, b):
    """boxes as [x1, y1, x2, y2]"""
    ix1, iy1 = max(a[0], b[0]), max(a[1], b[1])
    ix2, iy2 = min(a[2], b[2]), min(a[3], b[3])
    iw, ih = max(0.0, ix2-ix1), max(0.0, iy2-iy1)
```

```

inter = iw*ih
ua = (a[2]-a[0])*(a[3]-a[1]) + (b[2]-b[0])*(b[3]-b[1]) - inter
return inter/ua

# hand-checkable cases
print(f"identical boxes:      IoU = {iou([0,0,2,2],[0,0,2,2]):.3f}")
print(f"half-overlap:        IoU = {iou([0,0,2,2],[1,0,3,2]):.3f} (inter 2, union 6 -
> 1/3)")
print(f"corner quarter:      IoU = {iou([0,0,2,2],[1,1,3,3]):.3f} (inter 1, union 7 -
> 1/7)")
print(f"disjoint:            IoU = {iou([0,0,1,1],[2,2,3,3]):.3f}")

def nms(bboxes, scores, thr=0.5):
    order = np.argsort(scores)[::-1]; keep = []
    while len(order):
        i = order[0]; keep.append(i)
        rest = order[1:]
        order = rest[[iou(bboxes[i], bboxes[j]) < thr for j in rest]] # drop overlaps
    with the winner
    return keep

# cluster of noisy detections around 2 true objects + 1 outlier
true = [np.array([2,2,6,6]), np.array([8,3,12,9])]
boxes, scores = [], []
for t in true:
    for _ in range(6):
        boxes.append(t + rng.normal(0, .35, 4)); scores.append(rng.uniform(.6, .99))
boxes.append(np.array([13,10,15,12])); scores.append(0.30) # lone low-score
detection
boxes, scores = np.array(boxes), np.array(scores)
keep = nms(boxes, scores, 0.5)
print(f"{len(boxes)} raw detections -> NMS keeps {len(keep)} (indices {[int(i) for i
in keep]}))")

fig, axes = plt.subplots(1, 2, figsize=(9, 3.6))
for ax, sel, title in [(axes[0], range(len(boxes)), f"raw detections ({len(boxes)})"),
                        (axes[1], keep, f"after NMS @ IoU 0.5 ({len(keep)})")]:
    for i in sel:
        b = boxes[i]
        ax.add_patch(mp.Rectangle((b[0], b[1]), b[2]-b[0], b[3]-b[1], fill=False,
                                ec=plt.cm.viridis(scores[i]), lw=1.5))
    ax.set(xlim=(0, 16), ylim=(0, 13), title=title); ax.set_aspect('equal')
fig.tight_layout(); fig.savefig("figures/ch17/fig_l1_nms.png", dpi=150)
print("figure saved: fig_l1_nms.png")

```

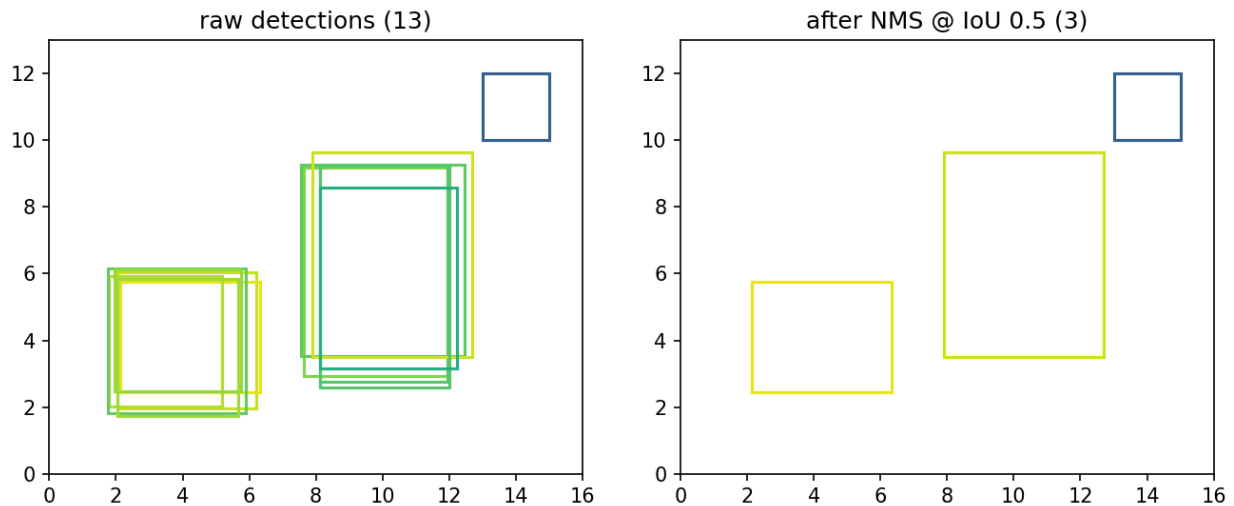
Output:

```

identical boxes:      IoU = 1.000
half-overlap:         IoU = 0.333 (inter 2, union 6 -> 1/3)
corner quarter:       IoU = 0.143 (inter 1, union 7 -> 1/7)
disjoint:             IoU = 0.000
13 raw detections -> NMS keeps 3 (indices [1, 9, 12])
figure saved: fig_l1_nms.png

```





## Listing 2 — Average precision and mAP, computed by hand

```

"""Listing 2: mAP from scratch -- the metric everyone quotes and few can compute."""
import numpy as np
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt

def iou(a, b):
    ix1, iy1 = max(a[0],b[0]), max(a[1],b[1]); ix2, iy2 = min(a[2],b[2]),
    min(a[3],b[3])
    inter = max(0.,ix2-ix1)*max(0.,iy2-iy1)
    return inter/((a[2]-a[0])*(a[3]-a[1]) + (b[2]-b[0])*(b[3]-b[1]) - inter)

def average_precision(dets, gts, iou_thr=0.5):
    """dets: list of (image_id, score, box); gts: {image_id: [box,...]} for ONE
    class"""
    n_gt = sum(len(v) for v in gts.values())
    dets = sorted(dets, key=lambda d: -d[1]) # by confidence, descending
    matched = {k: np.zeros(len(v), bool) for k, v in gts.items()}
    tp = np.zeros(len(dets)); fp = np.zeros(len(dets))
    for i, (img, score, box) in enumerate(dets):
        cands = gts.get(img, [])
        ious = [iou(box, g) for g in cands]
        j = int(np.argmax(ious)) if ious else -1
        if j >= 0 and ious[j] >= iou_thr and not matched[img][j]:
            tp[i] = 1; matched[img][j] = True # each GT may be claimed ONCE
        else:
            fp[i] = 1 # duplicate or bad-IoU or
    hallucination
    rec = np.cumsum(tp)/n_gt
    prec = np.cumsum(tp)/(np.cumsum(tp)+np.cumsum(fp))
    # all-point interpolation (COCO-style): precision envelope, integrate over recall
    mrec = np.concatenate([[0], rec, [1]]); mpre = np.concatenate([[1], prec, [0]])
    for k in range(len(mpre)-2, -1, -1): mpre[k] = max(mpre[k], mpre[k+1])
    idx = np.where(mrec[1:] != mrec[:-1])[0]
    return ((mrec[idx+1]-mrec[idx])*mpre[idx+1]).sum(), rec, prec, mpre, mrec

# toy set, hand-traceable: 5 GT boxes over 3 images; 7 detections incl one duplicate +
# one hallucination
G = {0: [[0,0,10,10],[20,20,30,30]], 1: [[5,5,15,15],[40,40,50,50]], 2: [[0,0,8,8]]}
D = [(0,.95,[0,0,10,10]), (0,.90,[21,21,31,31]), (1,.85,[6,6,16,16]),
      (0,.80,[1,1,11,11]), (2,.70,[60,60,70,70]), (1,.60,[40,41,50,49]),
      (2,.50,[0,1,8,9])]
ap, rec, prec, mpre, mrec = average_precision(D, G)
print("rank scores:", [f"{s:.2f}" for _,s,_ in sorted(D, key=lambda d: -d[1])])
print("recall:      ", np.round(rec, 3))

```

```

print("precision: ", np.round(prec, 3))
print(f"AP@0.5 = {ap:.4f}")
print("note: det 0.80 is a DUPLICATE of a matched GT -> counted FP; det 0.70 is a
hallucination -> FP")

# mAP = mean of per-class AP; show sensitivity to IoU threshold (COCO averages
0.50:0.95)
for thr in [0.5, 0.75, 0.9]:
    a, *_ = average_precision(D, G, thr)
    print(f"AP@{thr:.2f} = {a:.4f}")
fig, ax = plt.subplots(figsize=(5.2, 3.6))
ax.step(rec, prec, where='post', label='raw precision-recall')
ax.step(mrec, mpre, where='post', ls='--', label='interpolated envelope')
ax.set(xlabel='recall', ylabel='precision', title=f'AP@0.5 = {ap:.3f} (area under
envelope)')
ax.legend(); fig.tight_layout(); fig.savefig("figures/ch17/fig_l2_pr.png", dpi=150)
print("figure saved: fig_l2_pr.png")

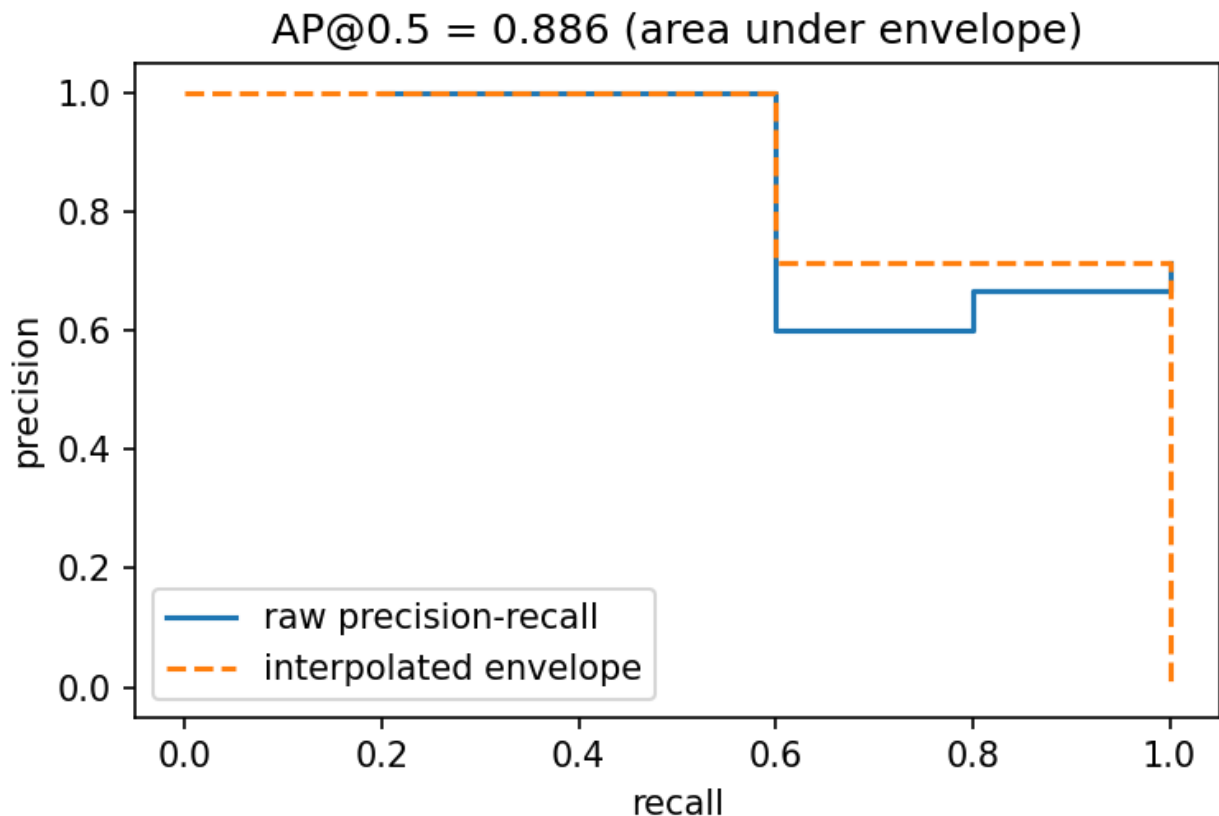
```

Output:

```

rank scores: ['0.95', '0.90', '0.85', '0.80', '0.70', '0.60', '0.50']
recall:      [0.2 0.4 0.6 0.6 0.6 0.8 1. ]
precision:   [1.   1.   1.   0.75 0.6  0.667 0.714]
AP@0.5 = 0.8857
note: det 0.80 is a DUPLICATE of a matched GT -> counted FP; det 0.70 is a
hallucination -> FP
AP@0.50 = 0.8857
AP@0.75 = 0.3714
AP@0.90 = 0.2000
figure saved: fig_l2_pr.png

```



## Listing 3 — Anchor boxes: assignment, encoding, and the exact decode round-trip

```

"""Listing 3: anchor boxes -- assignment, box encoding, and the exact decode round-trip."""
import numpy as np
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.patches as mp
rng = np.random.default_rng(3)

def iou(a, b):
    ix1, iy1 = max(a[0],b[0]), max(a[1],b[1]); ix2, iy2 = min(a[2],b[2]), min(a[3],b[3])
    inter = max(0.,ix2-ix1)*max(0.,iy2-iy1)
    return inter/((a[2]-a[0])*(a[3]-a[1]) + (b[2]-b[0])*(b[3]-b[1]) - inter)

# anchor grid: 4x4 centers on a 32x32 image, 3 shapes per center (square, wide, tall)
anchors = []
for cy in np.arange(4, 32, 8):
    for cx in np.arange(4, 32, 8):
        for (w, h) in [(8,8), (12,6), (6,12)]:
            anchors.append([cx-w/2, cy-h/2, cx+w/2, cy+h/2])
anchors = np.array(anchors)
print(f"anchor grid: {len(anchors)} anchors (4x4 centers x 3 aspect ratios)")

gt = np.array([[6,5,17,14], [20,18,29,30]]) # two ground-truth objects
M = np.array([[iou(a, g) for g in gt] for a in anchors]) # (n_anchors, n_gt)
best = M.argmax(0)
pos = set(int(b) for b in best) | set(np.where(M.max(1) > 0.5)[0].tolist())
neg = np.where(M.max(1) < 0.3)[0]
print(f"positives (best-per-GT + IoU>0.5): {sorted(pos)} negatives (IoU<0.3): {len(neg)}")
print(f" ignored (0.3-0.5): {len(anchors)-len(pos)-len(neg)}")

def encode(a, g):
    """Faster R-CNN parameterization: offsets relative to anchor size, log for scale"""
    aw, ah = a[2]-a[0], a[3]-a[1]; ax, ay = a[0]+aw/2, a[1]+ah/2
    gw, gh = g[2]-g[0], g[3]-g[1]; gx, gy = g[0]+gw/2, g[1]+gh/2
    return np.array([(gx-ax)/aw, (gy-ay)/ah, np.log(gw/aw), np.log(gh/ah)])

def decode(a, t):
    aw, ah = a[2]-a[0], a[3]-a[1]; ax, ay = a[0]+aw/2, a[1]+ah/2
    gx, gy = t[0]*aw+ax, t[1]*ah+ay; gw, gh = aw*np.exp(t[2]), ah*np.exp(t[3])
    return np.array([gx-gw/2, gy-gh/2, gx+gw/2, gy+gh/2])

worst = 0
for ai in pos:
    g = gt[M[ai].argmax()]
    t = encode(anchors[ai], g)
    worst = max(worst, np.abs(decode(anchors[ai], t) - g).max())
print(f"encode->decode round trip on all positive anchors: max abs error {worst:.2e}")
ai = sorted(pos)[0]
print(f"example target vector t = {np.round(encode(anchors[ai], gt[M[ai].argmax()]), 3)}")
print(f" (dx, dy in anchor-widths; dw, dh in log-scale)")

fig, ax = plt.subplots(figsize=(4.6, 4.6))
for a in anchors[:3]:
    # squares only, for legibility
    ax.add_patch(mp.Rectangle((a[0],a[1]), a[2]-a[0], a[3]-a[1], fill=False, ec='0.8', lw=.7))
for ai in pos:
    a = anchors[ai]
    ax.add_patch(mp.Rectangle((a[0],a[1]), a[2]-a[0], a[3]-a[1], fill=False, ec='tab:orange', lw=1.6))
for g in gt:

```

```

ax.add_patch(mp.Rectangle((g[0],g[1]), g[2]-g[0], g[3]-g[1], fill=False,
ec='tab:green', lw=2.2))
ax.set(xlim=(0,32), ylim=(0,32), title='anchors (grey), positives (orange), GT
(green)')
ax.set_aspect('equal'); fig.tight_layout(); fig.savefig("figures/ch17/
fig_l3_anchors.png", dpi=150)
print("figure saved: fig_l3_anchors.png")

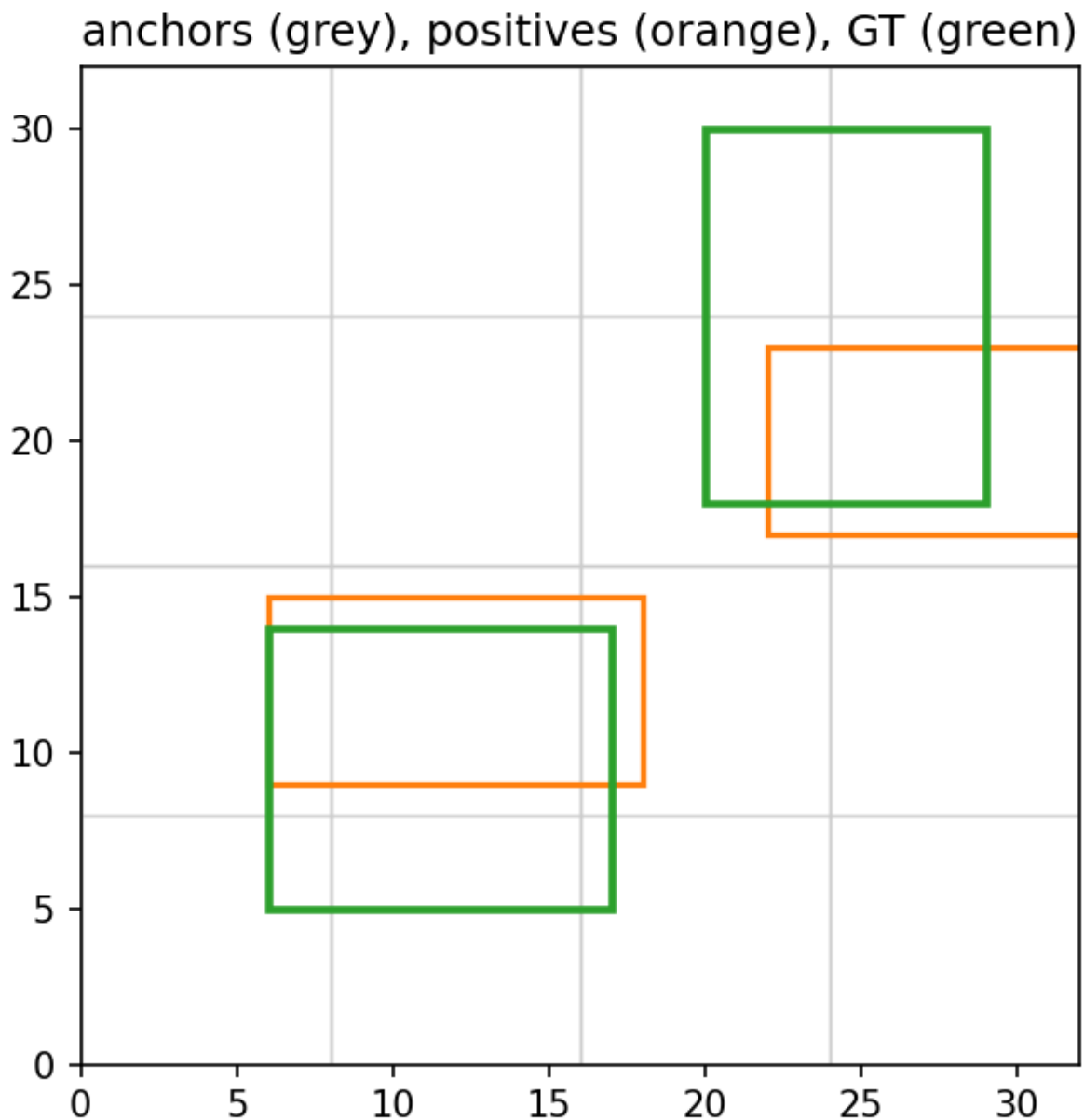
```

Output:

```

anchor grid: 48 anchors (4x4 centers x 3 aspect ratios)
positives (best-per-GT + IoU>0.5): [16, 34]   negatives (IoU<0.3): 45   ignored
(0.3-0.5): 1
encode->decode round trip on all positive anchors: max abs error 0.00e+00
example target vector t = [-0.042 -0.417 -0.087  0.405] (dx, dy in anchor-widths; dw,
dh in log-scale)
figure saved: fig_l3_anchors.png

```



## Listing 4 — A one-shot grid detector (YOLO's core idea), trained from scratch

```

"""Listing 4: a one-shot grid detector (YOLO's core idea) trained from scratch."""
import numpy as np
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.patches as mp
rng = np.random.default_rng(4)

S, IMG = 3, 24 # 3x3 grid over a 24x24 image
def make():
    """1-2 bright rectangles on noise; labels per cell: (obj, cx, cy, w, h), coords
cell-relative"""
    img = rng.normal(0, .15, (IMG, IMG)); lab = np.zeros((S, S, 5))
    for _ in range(rng.integers(1, 3)):
        w, h = rng.integers(4, 9, 2); x = rng.integers(0, IMG-w); y = rng.integers(0,
IMG-h)
        img[y:y+h, x:x+w] += 1.0
        cx, cy = (x+w/2)/IMG, (y+h/2)/IMG # center, in image fraction
        gi, gj = int(cx*S), int(cy*S) # responsible cell
        lab[gj, gi] = [1, cx*S-gi, cy*S-gj, w/IMG, h/IMG]
    return img, lab

D, Hd = IMG*IMG, 128
r = np.random.default_rng(0)
W1 = r.normal(0, (2/D)**.5, (D, Hd)); b1 = np.zeros(Hd)
W2 = r.normal(0, (2/Hd)**.5, (Hd, S*S*5)); b2 = np.zeros(S*S*5)
sig = lambda z: 1/(1+np.exp(-z))

def fwd(x):
    h = np.maximum(0, x@W1 + b1); o = sig(h@W2 + b2).reshape(S, S, 5)
    return h, o
def step(img, lab, lr):
    global W1, b1, W2, b2
    x = img.ravel(); h, o = fwd(x)
    obj = lab[..., 0:1]
    # gradient wrt PRE-sigmoid logits: BCE on obj -> (o-y); box SSE only where obj=1
    d = np.zeros_like(o)
    d[..., 0:1] = o[..., 0:1] - obj
    d[..., 1:] = 5.0 * obj * (o[..., 1:] - lab[..., 1:]) * o[..., 1:]*(1-o[..., 1:])
    d = d.reshape(-1)
    dW2 = np.outer(h, d); db2 = d
    dh = (d@W2.T)*(h > 0)
    W2 -= lr*dW2; b2 -= lr*db2; W1 -= lr*np.outer(x, dh); b1 -= lr*dh

for it in range(30000):
    img, lab = make(); step(img, lab, 0.02 if it < 20000 else 0.005)

def decode(o, thr=.5):
    out = []
    for gj in range(S):
        for gi in range(S):
            p, cx, cy, w, h = o[gj, gi]
            if p > thr:
                cx, cy = (gi+cx)/S*IMG, (gj+cy)/S*IMG; w, h = w*IMG, h*IMG
                out.append((p, [cx-w/2, cy-h/2, cx+w/2, cy+h/2]))
    return out
def iou(a, b):
    ix1, iy1 = max(a[0],b[0]), max(a[1],b[1]); ix2, iy2 = min(a[2],b[2]),
min(a[3],b[3])
    inter = max(0.,ix2-ix1)*max(0.,iy2-iy1)
    return inter/((a[2]-a[0])*(a[3]-a[1]) + (b[2]-b[0])*(b[3]-b[1]) - inter)

hits, ious, n_gt, n_pred = 0, [], 0, 0
for _ in range(300):

```

```

img, lab = make(); _, o = fwd(img.ravel())
dets = decode(o); n_pred += len(dets)
for gj in range(S):
    for gi in range(S):
        if lab[gj, gi, 0]:
            n_gt += 1
            cx, cy = (gi+lab[gj,gi,1])/S*IMG, (gj+lab[gj,gi,2])/S*IMG
            w, h = lab[gj,gi,3]*IMG, lab[gj,gi,4]*IMG
            g = [cx-w/2, cy-h/2, cx+w/2, cy+h/2]
            best = max([iou(d[1], g) for d in dets], default=0)
            if best >= .5: hits += 1; ious.append(best)
print(f"grid detector: recall@0.5 {hits/n_gt:.3f} precision {hits/max(1,n_pred):.3f}"
      f" mean IoU of matches {np.mean(ious):.3f} ({n_gt} objects, {n_pred}
predictions)")

fig, axes = plt.subplots(1, 3, figsize=(9.6, 3.4))
for ax in axes:
    img, lab = make(); _, o = fwd(img.ravel())
    ax.imshow(img, cmap='gray'); ax.set_xticks([]); ax.set_yticks([])
    for p, b in decode(o):
        ax.add_patch(mp.Rectangle((b[0], b[1]), b[2]-b[0], b[3]-b[1], fill=False,
ec='r', lw=1.8))
fig.suptitle('grid-detector predictions (red) on unseen images')
fig.tight_layout(); fig.savefig("figures/ch17/fig_l4_grid.png", dpi=150)
print("figure saved: fig_l4_grid.png")

```

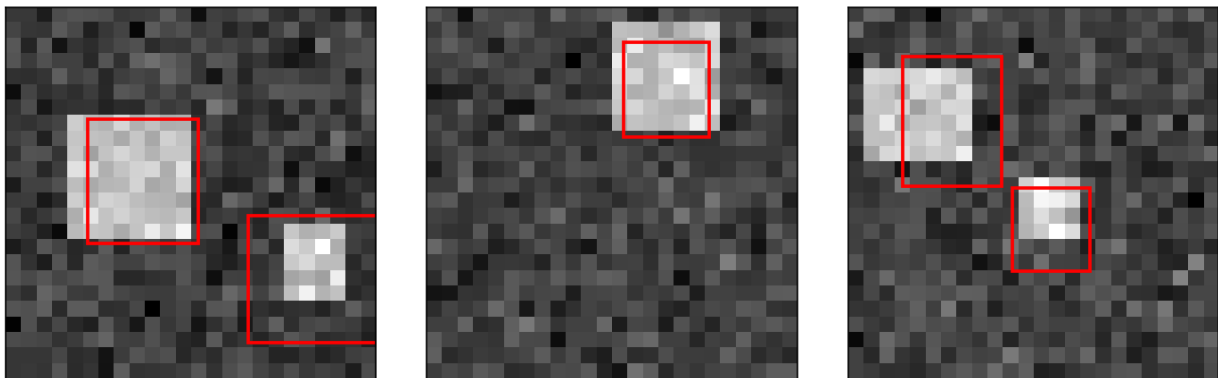
Output:

```

grid detector: recall@0.5 0.811 precision 0.828 mean IoU of matches 0.675 (428
objects, 419 predictions)
figure saved: fig_l4_grid.png

```

grid-detector predictions (red) on unseen images



## Listing 5 — A miniature U-Net: what skip connections buy at the boundary

```

"""Listing 5: a miniature U-Net -- what skip connections buy at the boundary."""
import numpy as np
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(5)
N = 24

def make():
    """image: noisy disks; mask: their exact pixels"""
    img = rng.normal(0, .2, (N, N)); msk = np.zeros((N, N))

```

```

yy, xx = np.mgrid[:N, :N]
for _ in range(rng.integers(1, 3)):
    cy, cx, rad = rng.uniform(5, 19), rng.uniform(5, 19), rng.uniform(3, 6)
    d = (yy-cy)**2 + (xx-cx)**2 <= rad**2
    img[d] += 1.0; msk[d] = 1
return img[None], msk # (1, N, N), (N, N)

def im2col(x, k=3):
    C, H, W = x.shape; p = k//2
    xp = np.pad(x, ((0,0),(p,p),(p,p)))
    cols = np.empty((C*k*k, H*W))
    i = 0
    for c in range(C):
        for dy in range(k):
            for dx in range(k):
                cols[i] = xp[c, dy:dy+H, dx:dx+W].ravel(); i += 1
    return cols

def conv_f(x, W, b): # W: (Cout, Cin*9)
    cols = im2col(x); out = (W@cols + b[:, None]).reshape(len(W), *x.shape[1:])
    return out, cols

def conv_b(dout, cols, W, xshape):
    Cout = len(W); d2 = dout.reshape(Cout, -1)
    dW = d2@cols.T; db = d2.sum(1)
    dcols = W.T@d2 # (Cin*9, HW)
    C, H, W_ = xshape; p = 1; dxp = np.zeros((C, H+2, W_+2))
    i = 0
    for c in range(C):
        for dy in range(3):
            for dx in range(3):
                dxp[c, dy:dy+H, dx:dx+W_] += dcols[i].reshape(H, W_); i += 1
    return dW, db, dxp[:, 1:-1, 1:-1]

def pool_f(x): # 2x2 mean pool
    C, H, W = x.shape; return x.reshape(C, H//2, 2, W//2, 2).mean((2, 4))

def pool_b(d):
    return np.repeat(np.repeat(d, 2, -1), 2, -2)/4
up = lambda x: np.repeat(np.repeat(x, 2, -1), 2, -2) # nearest-neighbor upsample
up_b = lambda d: d.reshape(d.shape[0], d.shape[1]//2, 2, d.shape[2]//2, 2).sum((2, 4))
sig = lambda z: 1/(1+np.exp(-z))

def train(skip, iters=4000, lr=0.05):
    r = np.random.default_rng(0)
    P = dict(W1=r.normal(0, .3, (8,9)), b1=np.zeros(8), # conv 1->8 @24
             W2=r.normal(0, .15, (16,72)), b2=np.zeros(16), # conv 8->16 @12
             W3=r.normal(0, .1, (8,144)), b3=np.zeros(8), # conv 16->8 @24 (after
up)
             Wo=r.normal(0, .2, (1,(16 if skip else 8)*9)), bo=np.zeros(1))
    def fwd(x):
        c = {}
        a1, c['c1'] = conv_f(x, P['W1'], P['b1']); r1 = np.maximum(0, a1) #
(8,24,24)
        p1 = pool_f(r1) #
(8,12,12)
        a2, c['c2'] = conv_f(p1, P['W2'], P['b2']); r2 = np.maximum(0, a2) #
(16,12,12)
        u2 = up(r2) #
(16,24,24)
        a3, c['c3'] = conv_f(u2, P['W3'], P['b3']); r3 = np.maximum(0, a3) #
(8,24,24)
        feat = np.concatenate([r3, r1]) if skip else r3
        o, c['co'] = conv_f(feat, P['Wo'], P['bo'])
        c.update(a1=a1, r1=r1, p1=p1, a2=a2, r2=r2, u2=u2, a3=a3, r3=r3, feat=feat,
x=x)
    return sig(o[0]), c

def step(x, y):
    pr, c = fwd(x)
    d = ((pr - y)/y.size)[None] # BCE grad at logits
    dWo, dbo, dfeat = conv_b(d, c['co'], P['Wo'], c['feat'].shape)
    dr3 = dfeat[:8]; dr1_skip = dfeat[8:] if skip else 0

```

```

da3 = dr3*(c['a3'] > 0)
dW3, db3, du2 = conv_b(da3, c['c3'], P['W3'], c['u2'].shape)
dr2 = up_b(du2); da2 = dr2*(c['a2'] > 0)
dW2, db2, dp1 = conv_b(da2, c['c2'], P['W2'], c['p1'].shape)
dr1 = pool_b(dp1) + dr1_skip
da1 = dr1*(c['a1'] > 0)
dW1, db1, _ = conv_b(da1, c['c1'], P['W1'], c['x'].shape)
for k, g in [(('W1',dW1),('b1',db1),('W2',dW2),('b2',db2),('W3',dW3),('b3',db3),
('Wo',dWo),('bo',dbo))]:
    P[k] -= lr*g
for _ in range(iters):
    step(*make())
dice, bnd = [], []
for _ in range(150):
    x, y = make(); pr, _ = fwd(x); pb = pr > .5
    dice.append(2*(pb*y).sum()/(pb.sum()+y.sum()+1e-9))
    edge = (np.abs(np.diff(y, axis=0, prepend=0)) + np.abs(np.diff(y, axis=1,
prepend=0))) > 0
    bnd.append((pb == y)[edge].mean()) # accuracy ON boundary
pixels only
return np.mean(dice), np.mean(bnd), fwd

for skip in [False, True]:
    dice, bnd, fwd = train(skip)
    print(f"skip={str(skip):5s}: Dice {dice:.3f}    boundary-pixel accuracy {bnd:.3f}")

x, y = make()
_, _, fwdN = train(False); _, _, fwdS = train(True)
fig, axes = plt.subplots(1, 4, figsize=(11, 3))
for ax, im, t in zip(axes, [x[0], y, fwdN(x)[0] > .5, fwdS(x)[0] > .5],
['input', 'ground truth', 'no skip', 'with skip']):
    ax.imshow(im, cmap='gray'); ax.set_title(t); ax.set_xticks([]); ax.set_yticks([])
fig.tight_layout(); fig.savefig("figures/ch17/fig_l5_unet.png", dpi=150)
print("figure saved: fig_l5_unet.png")

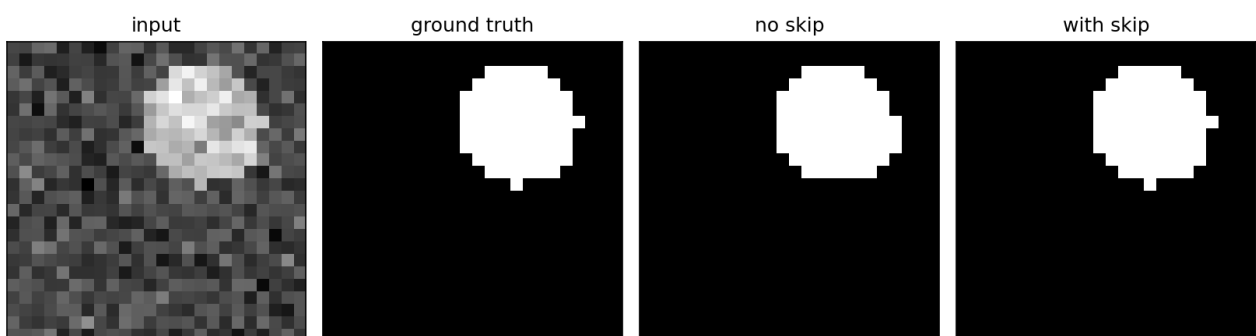
```

Output:

```

skip=False: Dice 0.983    boundary-pixel accuracy 0.941
skip=True : Dice 0.993    boundary-pixel accuracy 0.976
figure saved: fig_l5_unet.png

```



## Listing 6 — A Vision Transformer from scratch: patches are tokens

```

"""Listing 6: a Vision Transformer from scratch -- patches are tokens."""
import numpy as np
from sklearn.datasets import load_digits
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(6)

```



```

X, y = load_digits(return_X_y=True); X = X.reshape(-1, 8, 8)/16.0
idx = rng.permutation(len(X)); tr, te = idx[:1400], idx[1400:]

P, d, H, depth, C = 2, 32, 4, 2, 10; dh = d//H
npat = (8//P)**2
# 16 patches -> 16 tokens (+1 CLS)
n = npat + 1
def patchify(img):
    # (8,8) -> (16, 4)
    return img.reshape(8//P, P, 8//P, P).transpose(0,2,1,3).reshape(npat, P*P)

def softmax(z):
    z = z - z.max(-1, keepdims=True); e = np.exp(z); return e / e.sum(-1,
keepdims=True)
def ln_f(x):
    mu = x.mean(-1, keepdims=True); xc = x - mu
    inv = 1/np.sqrt((xc**2).mean(-1, keepdims=True)+1e-6); return xc*inv, (xc, inv)
def ln_b(dy, c):
    xc, inv = c
    return inv*(dy - dy.mean(-1, keepdims=True) - xc*inv**2*(dy*xc).mean(-1,
keepdims=True))

r = np.random.default_rng(0)
params = dict(Wp=r.normal(0, .5, (P*P,d)), cls=r.normal(0, .5,d), pos=r.normal(0, .1,
(n,d)),
              Wu=r.normal(0,d**-.5, (d,C)))
for i in range(depth):
    for w, shp, sc in [('Wq', (d,d),d**-.5), ('Wk', (d,d),d**-.5), ('Wv', (d,d),d**-.5),
                      ('Wo', (d,d),d**-.5), ('W1', (d,4*d),d**-.5), ('W2', (4*d,d),
(4*d)**-.5)]:
        params[f'{w}{i}'] = r.normal(0, sc, shp)

def fwd(img):
    X0 = np.vstack([params['cls'], patchify(img)@params['Wp']]) + params['pos']
    Xc, caches = X0, []
    for i in range(depth):
        c = {}
        h, c['ln1'] = ln_f(Xc)
        Q = (h@params[f'Wq{i}']).reshape(n,H,dh); K =
(h@params[f'Wk{i}']).reshape(n,H,dh)
        Vv = (h@params[f'Wv{i}']).reshape(n,H,dh)
        A = np.stack([softmax(Q[:,hh]@K[:,hh].T/np.sqrt(dh)) for hh in range(H)])
# NO mask: encoder
        ao = np.stack([A[hh]@Vv[:,hh] for hh in range(H)], 1).reshape(n,d)
        c['at'] = (h,Q,K,Vv,A,ao); X1 = Xc + ao@params[f'Wo{i}']
        h2, c['ln2'] = ln_f(X1); z = h2@params[f'W1{i}']; rr = np.maximum(0,z)
        c['ffn'] = (h2,z,rr); Xc = X1 + rr@params[f'W2{i}']; caches.append(c)
    hf, lnf = ln_f(Xc)
    return hf[0]@params['Wu'], caches, lnf, hf, img # classify from the CLS token

def step(img, yi):
    logits, caches, lnf, hf, img = fwd(img)
    Pr = softmax(logits); L = -np.log(Pr[yi]+1e-12)
    g = {k: np.zeros_like(v) for k, v in params.items()}
    dlog = Pr.copy(); dlog[yi] -= 1
    dhf = np.zeros_like(hf); dhf[0] = params['Wu']@dlog
    g['Wu'] += np.outer(hf[0], dlog); dX = ln_b(dhf, lnf)
    for i in range(depth-1, -1, -1):
        c = caches[i]; h2, z, rr = c['ffn']
        drr = dX@params[f'W2{i}'].T; g[f'W2{i}'] += rr.T@dX
        dz = drr*(z>0); g[f'W1{i}'] += h2.T@dz
        dX1 = dX + ln_b(dz@params[f'W1{i}'].T, c['ln2'])
        h,Q,K,Vv,A,ao = c['at']
        dao = (dX1@params[f'Wo{i}']).T.reshape(n,H,dh); g[f'Wo{i}'] += ao.T@dX1
        dhid = np.zeros_like(h); dQ = np.zeros_like(Q); dK = np.zeros_like(K); dV =
np.zeros_like(Vv)
        for hh in range(H):
            dA = dao[:,hh]@Vv[:,hh].T; dV[:,hh] = A[hh].T@dA[:,hh]

```

```

        dS = A[hh]*(dA-(A[hh]*dA).sum(-1,keepdims=True))
        dQ[:,hh] = dS@K[:,hh]/np.sqrt(dh); dK[:,hh] = dS.T@Q[:,hh]/np.sqrt(dh)
    for nm, dM in [ ('Wq',dQ), ('Wk',dK), ('Wv',dV) ]:
        g[f'{nm}{i}'] += h.T@dM.reshape(n,d); dhid += dM.reshape(n,d)@params[f'{nm}{i}'].T
    dX = dX1 + ln_b(dhid, c['ln1'])
    g['pos'] += dX; g['cls'] += dX[0]; g['Wp'] += patchify(img).T@dX[1:]
    return L, g

# gradient check across every parameter family
img, yi = X[tr[0]], y[tr[0]]; _, g0 = step(img, yi); eps, worst = 1e-5, 0
for key, idx2 in [ ('Wq0', (3,5)), ('W21', (40,7)), ('Wp', (2,9)), ('pos', (5,3)), ('cls', (4,)), ('Wu', (10,3)) ]:
    W = params[key]; keep = W[idx2]
    W[idx2] = keep+eps; Lp, _ = step(img, yi); W[idx2] = keep-eps; Lm, _ = step(img, yi);
    W[idx2] = keep
    num = (Lp-Lm)/(2*eps); worst = max(worst, abs(num-g0[key][idx2])/max(1e-12, abs(num)+abs(g0[key][idx2])))
print(f"ViT gradient check (patch-embed/pos/CLS/attn/FFN/head): worst rel err {worst:.2e}")

mom = {k: np.zeros_like(v) for k in params for k, v in [(k, params[k])]}
vel = {k: np.zeros_like(v) for k, v in params.items()}
b1, b2, lr, t = 0.9, 0.999, 1e-3, 0
for ep in range(3):
    for i in rng.permutation(tr):
        t += 1; L, g = step(X[i], y[i])
        for k in params:
            mom[k] = b1*mom[k]+(1-b1)*g[k]; vel[k] = b2*vel[k]+(1-b2)*g[k]**2
            params[k] -= lr*(mom[k]/(1-b1**t))/(np.sqrt(vel[k]/(1-b2**t))+1e-8)
acc = np.mean([fwd(X[i])[0].argmax() == y[i] for i in te])
print(f"ViT (2 blocks, 4 heads, 16 patch-tokens + CLS) test accuracy: {acc:.3f}")

# what does CLS look at? mean over heads of last block's CLS-row attention
fig, axes = plt.subplots(1, 4, figsize=(10, 2.9))
for k, ax in zip(te[:4], axes):
    _, caches, _, _ = fwd(X[k])
    att = caches[-1]['at'][4].mean(0)[0, 1:].reshape(4, 4) # CLS row, patch grid
    ax.imshow(X[k], cmap='gray'); ax.set_xticks([]); ax.set_yticks([])
    ax.imshow(np.kron(att, np.ones((2,2))), cmap='hot', alpha=.45)
    ax.set_title(f"digit {y[k]}")
fig.suptitle('CLS-token attention over patches (last block, head-mean)')
fig.tight_layout(); fig.savefig("figures/ch17/fig_l6_vit.png", dpi=150)
print("figure saved: fig_l6_vit.png")

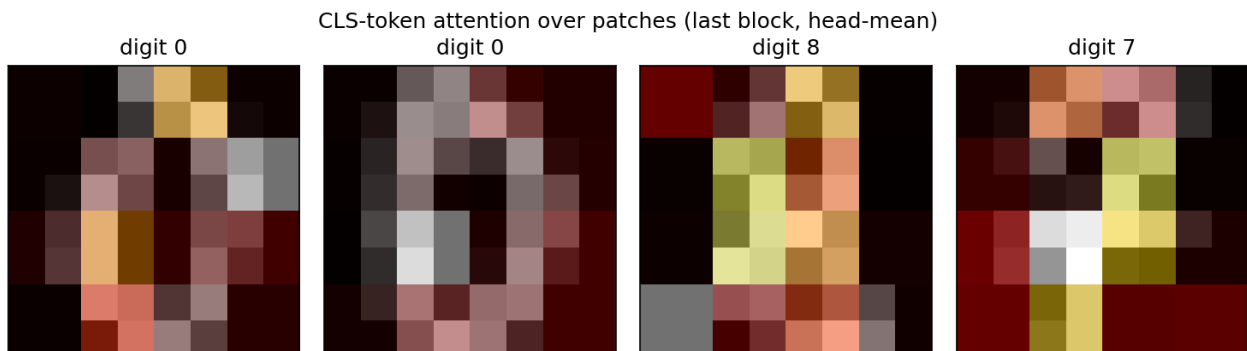
```

Output:

```

ViT gradient check (patch-embed/pos/CLS/attn/FFN/head): worst rel err 1.64e-10
ViT (2 blocks, 4 heads, 16 patch-tokens + CLS) test accuracy: 0.924
figure saved: fig_l6_vit.png

```



## Listing 7 — CLIP in miniature: contrastive two-tower training

```

"""Listing 7: CLIP in miniature -- contrastive two-tower training, classification as
retrieval."""
import numpy as np
from sklearn.datasets import load_digits
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(7)

X, y = load_digits(return_X_y=True); X = X/16.0
idx = rng.permutation(len(X)); tr, te = idx[:1400], idx[1400:]
D, E, C, B = 64, 16, 10, 32

def softmax(z):
    z = z - z.max(-1, keepdims=True); e = np.exp(z); return e/e.sum(-1, keepdims=True)
def norm_f(v):
    nv = np.linalg.norm(v, axis=-1, keepdims=True); return v/nv, nv

def train(learn_temp, iters=3000, lr=0.05):
    r = np.random.default_rng(0)
    Wi = r.normal(0, D**-.5, (D, E)) # image tower: one linear
    layer
    Wt = r.normal(0, 1, (C, E))
    # text tower: an embedding per caption
    logt = np.log(1/0.07) if learn_temp else 0.0 # CLIP's learnable log-
    temperature
    for it in range(iters):
        b = rng.choice(tr, B, replace=False)
        yi = y[b]
        I0 = X[b]@Wi; T0 = Wt[yi]
        I, nI = norm_f(I0); T, nT = norm_f(T0) # unit sphere
        s = np.exp(logt)
        L1 = s*(I@T.T) # image->text logits (B,B)
        P1 = softmax(L1); P2 = softmax(L1.T) # text->image
        tgt = np.arange(B)
        # symmetric InfoNCE; NOTE in-batch DUPLICATE captions make off-diagonal
    "negatives" false
        dL1 = (P1 - np.eye(B))/(2*B) + ((P2 - np.eye(B))/(2*B)).T
        dI = s*(dL1@T); dT = s*(dL1.T@I)
        if learn_temp:
            logt -= lr*(dL1*(I@T.T)).sum()*s
            dI0 = (dI - I*(dI*I).sum(-1, keepdims=True))/nI # backward through normalize
            dT0 = (dT - T*(dT*T).sum(-1, keepdims=True))/nT
            Wi -= lr*(X[b].T@dI0)
            np.add.at(Wt, yi, -lr*dT0)
        # zero-shot-style classification: nearest caption embedding
        Iemb = norm_f(X[te]@Wi)[0]; Temb = norm_f(Wt)[0]
        acc = (np.argmax(Iemb@Temb.T, 1) == y[te]).mean()
        return acc, Iemb, Temb, np.exp(logt)

for lt in [False, True]:
    acc, Iemb, Temb, temp = train(lt)
    print(f"learned temperature={str(lt):5s}: retrieval accuracy {acc:.3f} final
    scale {temp:6.2f}")

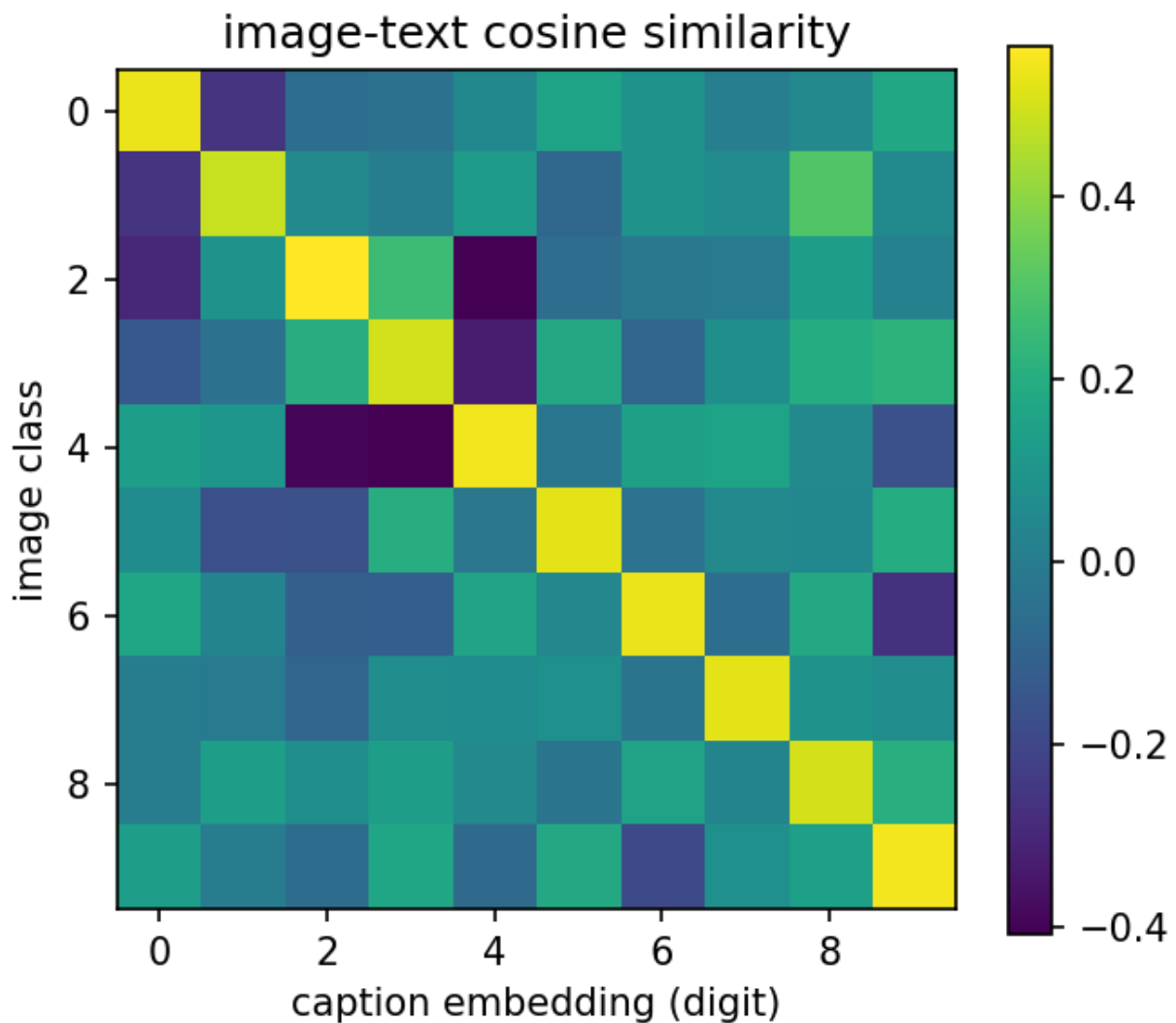
sim = np.zeros((C, C)) # class-mean image emb vs
caption
for c in range(C):
    sim[c] = Iemb[y[te] == c].mean(0)@Temb.T
fig, ax = plt.subplots(figsize=(4.6, 4))
im = ax.imshow(sim, cmap='viridis'); plt.colorbar(im)
ax.set(xlabel='caption embedding (digit)', ylabel='image class', title='image-text
cosine similarity')

```

```
fig.tight_layout(); fig.savefig("figures/ch17/fig_l7_clip.png", dpi=150)
print("figure saved: fig_l7_clip.png")
```

Output:

```
learned temperature=False: retrieval accuracy 0.904    final scale    1.00
learned temperature=True : retrieval accuracy 0.937    final scale   17.44
figure saved: fig_l7_clip.png
```



## Listing 8 — Augmentation as a claim about label-invariance -- and when the claim is false

```
"""Listing 8: augmentation is a statement about label-invariance -- and it can be
false."""
import numpy as np
from sklearn.datasets import load_digits
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
rng = np.random.default_rng(8)

X, y = load_digits(return_X_y=True); X = (X/16.0).reshape(-1, 8, 8)
```

```

idx = rng.permutation(len(X)); tr, te = idx[:300], idx[1400:]      # SMALL train set:
aug should matter

def shift(img):
    dy, dx = rng.integers(-1, 2, 2)
    out = np.zeros_like(img)                                     # zero-pad shift (np.roll
would wrap pixels around)
    src_y = slice(max(0, -dy), 8-max(0, dy)); dst_y = slice(max(0, dy), 8-max(0, -dy))
    src_x = slice(max(0, -dx), 8-max(0, dx)); dst_x = slice(max(0, dx), 8-max(0, -dx))
    out[dst_y, dst_x] = img[src_y, src_x]
    return out
flip = lambda img: img[:, ::-1]

def train(aug, iters=15000, lr=0.05):
    r = np.random.default_rng(0)
    W1 = r.normal(0, (2/64)**.5, (64, 64)); b1 = np.zeros(64)
    W2 = r.normal(0, (2/64)**.5, (64, 10)); b2 = np.zeros(10)
    def fwd(x):
        h = np.maximum(0, x@W1+b1); z = h@W2+b2
        e = np.exp(z-z.max()); return h, e/e.sum()
    for it in range(iters):
        i = tr[rng.integers(len(tr))]
        img = aug(X[i]) if (aug and rng.random() < .5) else X[i]
# MIX augmented with originals
        x = img.ravel(); h, p = fwd(x)
        d = p.copy(); d[y[i]] -= 1
        dW2 = np.outer(h, d); dh = (W2@d)*(h > 0)
        W2 -= lr*dW2; b2 -= lr*d; W1 -= lr*np.outer(x, dh); b1 -= lr*dh
    clean = np.mean([fwd(X[i].ravel())[1].argmax() == y[i] for i in te])
    jit = np.mean([fwd(shift(X[i]).ravel())[1].argmax() == y[i] for i in te])
    return clean, jit

both = lambda im: flip(shift(im)) if rng.random() < .5 else shift(im)
for name, aug in [("none", None), ("+-lpx shifts", shift), ("horizontal flip", flip),
("shift+flip", both)]:
    c, j = train(aug)
    print(f"augmentation {name:16s}: clean test {c:.3f}    jittered test {j:.3f}")
print("shift encodes a TRUE invariance (digit identity survives translation);")
print("flip encodes a FALSE one (a flipped 2 is not a 2) -- augmentation injects
assumptions, not free data")

fig, axes = plt.subplots(2, 5, figsize=(9, 3.6))
for c, ax0, ax1 in zip(range(0, 10, 2), axes[0], axes[1]):
    k = te[np.where(y[te] == c)[0][0]]
    ax0.imshow(X[k], cmap='gray'); ax0.set_title(f"digit {c}")
    ax1.imshow(flip(X[k]), cmap='gray'); ax1.set_title("flipped")
    for ax in (ax0, ax1): ax.set_xticks([]); ax.set_yticks([])
fig.tight_layout(); fig.savefig("figures/ch17/fig_l8_aug.png", dpi=150)
print("figure saved: fig_l8_aug.png")

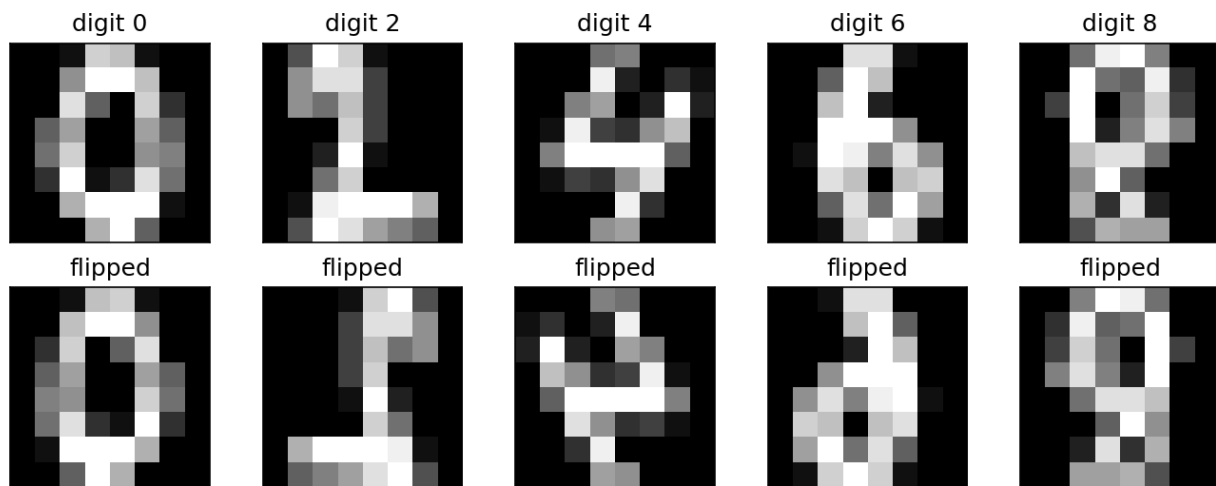
```

Output:

```

augmentation none           : clean test 0.962    jittered test 0.343
augmentation +-lpx shifts   : clean test 0.950    jittered test 0.872
augmentation horizontal flip : clean test 0.924    jittered test 0.343
augmentation shift+flip     : clean test 0.945    jittered test 0.768
shift encodes a TRUE invariance (digit identity survives translation);
flip encodes a FALSE one (a flipped 2 is not a 2) -- augmentation injects
assumptions, not free data
figure saved: fig_l8_aug.png

```



## Interview questions and answers

### Q1. Define IoU and give the values for identical, half-overlapping, and disjoint unit boxes.

$\text{IoU} = \text{area of intersection} / \text{area of union}$ . Identical: 1.0. Two 2x2 boxes offset by half: intersection 2, union  $8-2=6$ , IoU  $1/3$ . Corner quarter-overlap of 2x2 boxes:  $1/7$  (inter 1, union 7). Disjoint: 0. Implementation detail (Listing 1): intersection corners are (max of mins, min of maxes) with widths clamped at 0; union =  $\text{areaA} + \text{areaB} - \text{intersection}$  — forgetting the subtraction double-counts. Thresholds in practice: 0.5 = classic detection hit, 0.75 = tight, COCO averages 0.50:0.95.

### Q2. Write the NMS algorithm. What is its failure mode and the remedies?

Sort detections by confidence; keep the top one; remove all remaining boxes with IoU above threshold against it; repeat on the survivors (Listing 1: 13 noisy detections  $\rightarrow$  3). Per class, usually. Failure: two GENUINELY overlapping same-class objects (crowds, occlusion) — the weaker true detection is deleted, capping recall; mAP then counts the lost object as a miss. Remedies: Soft-NMS (decay overlapping scores instead of deleting), higher threshold + downstream filtering, learned NMS, or DETR's set prediction with bipartite matching, which needs no NMS at all.

**Q3. Walk through computing AP for one class, exactly.**

Sort all detections across images by confidence. For each, match to the best unclaimed same-image GT: if  $\text{IoU} \geq \text{threshold}$  and that GT is unclaimed, TP and claim it; else FP (this is how duplicates are punished — the second detection of a claimed object is an FP even at perfect IoU; Listing 2's 0.80-scored duplicate). Cumulative precision/recall down the ranking; make precision monotone from the right (the envelope); AP = area under the envelope over recall. Listing 2: seven detections, five GT, one duplicate + one hallucination  $\rightarrow$  AP 0.886. mAP = mean over classes; COCO additionally means over IoU 0.50:0.95 (the listing: 0.886/0.371/0.200 at 0.5/0.75/0.9).

**Q4. Why does mAP integrate over the precision-recall curve rather than report one operating point?**

A detector's score threshold is a free parameter; any single P/R point rewards whoever tuned their threshold to the test set. Integrating over the whole ranking evaluates the QUALITY OF THE ORDERING — every threshold at once — the same argument as ROC-AUC vs accuracy (Chapter 10). The envelope interpolation exists because raw precision zigzags as FPs land between TPs; taking the running max says "at this recall you could have had at least this precision by thresholding higher." Corollary worth stating: mAP is invariant to score calibration (only order matters), so a great-mAP detector can still have garbage confidence values — calibrate separately if downstream logic consumes them.

**Q5. Explain anchor boxes: why they exist and how targets are encoded.**

Regressing arbitrary boxes from dense features is unstable — outputs span position and scale nonlinearly. Anchors fix a reference lattice (positions  $\times$  scales  $\times$  aspect ratios); the network predicts small CORRECTIONS to the nearest reference:  $dx = (gx - ax)/aw$ ,  $dy = (gy - ay)/ah$  (offsets in anchor-widths),  $dw = \log(gw/aw)$ ,  $dh = \log(gh/ah)$  (log makes scale multiplicative and symmetric). All targets  $O(1)$  regardless of object size — learnable by an L1/L2 head. Listing 3: exact encode  $\rightarrow$  decode round-trip, error 0.0. Assignment: positive if best-for-a-GT or  $\text{IoU} > 0.5$ , negative below 0.3, IGNORE 0.3-0.5 — ambiguous anchors are noise, not signal.

**Q6. Tell the R-CNN -> Fast -> Faster -> Mask story as an efficiency argument.**

R-CNN: ~2k selective-search proposals, each warped through the CNN separately — thousands of passes per image, minutes each. Fast R-CNN: backbone runs ONCE; RoI pooling crops proposal features from the shared map — convolution deduplicated, but proposals still external and slow. Faster R-CNN: the RPN — a conv head scoring objectness + box offsets over anchors at every feature-map position — brings proposals inside the network; end-to-end trainable, near-real-time. Mask R-CNN: add a per-RoI mask branch and replace RoI pooling's coordinate ROUNDING with RoIAlign's bilinear sampling — quantization error is harmless for classification, fatal for pixel-accurate masks. Each step removes exactly one redundancy; being able to name which one is the interview answer.

**Q7. One-stage vs two-stage detectors: the real trade and what fixed one-stage accuracy.**

Two-stage: propose (~1k candidates), then classify/refine each — a focus mechanism that balances classes before the expensive head. One-stage (YOLO, SSD, RetinaNet): predict densely over all positions/anchors in one pass — faster, simpler, but scoring ~100k locations of which ~10 are objects: extreme easy-negative imbalance that swamps cross-entropy. Focal loss  $(1-p_t)^\gamma$  (Chapter 12: hard:easy gradient ratio pushed from 23x to 4600x at gamma 2) down-weights the easy ocean; RetinaNet with it matched two-stage mAP. Modern practice: one-stage (+FPN, +better assignment) dominates real-time; two-stage persists where max accuracy on small/crowded objects matters.



**Q8. Explain YOLO's grid formulation. What did Listing 4's miniature implement?**

Divide the image into  $S \times S$  cells; the cell containing an object's CENTER is responsible for it, predicting objectness, center offset within the cell (sigmoid  $\rightarrow [0,1]$ ), and box size (as image fractions; real YOLO uses anchor-relative). Loss: BCE on objectness everywhere, box regression only on responsible cells (masking non-object cells out of the box loss is what makes it work). Listing 4:  $3 \times 3$  grid, per-cell 5-vector, hand backward, 30k SGD steps  $\rightarrow$  recall 0.81 / precision 0.83 / matched IoU 0.68 on synthetic scenes. Known YOLO weaknesses inherit from the formulation: one owner per cell hurts small clustered objects; coarse grids hurt localization — later versions add anchors, multi-scale grids, better assignment.

**Q9. What is DETR and why does it need neither anchors nor NMS?**

DETR feeds backbone features (+ positional encodings) to a transformer;  $N$  learned OBJECT QUERIES cross-attend into them and each emits (class, box) — detection as direct SET prediction. Training uses Hungarian matching: a bipartite assignment between predictions and GT minimizing class+box cost; each GT matches exactly one query, all unmatched queries train toward "no object." Deduplication is therefore built into the loss — two queries cannot both claim one GT, so the redundancy NMS existed to clean never arises; anchors are unnecessary because queries learn their own spatial specializations. Costs: slow convergence (deformable-attention DETRs fix), original weakness on small objects, and  $N$  caps detections per image.

**Q10. Semantic vs instance vs panoptic segmentation — definitions and canonical architectures.**

Semantic: every pixel gets a class; instances merge (two dogs = one dog-blob) — FCN/U-Net/DeepLab lineage, dense per-pixel classification. Instance: separate mask per object, background ignored — detection-based (Mask R-CNN: box, then binary mask within the RoI). Panoptic: unified — every pixel classed, countable "things" get instance IDs, amorphous "stuff" (sky, road) class only; architectures fuse a semantic branch and an instance branch, or use mask-transformer heads (Mask2Former) that emit both. Choosing which task the product actually needs (pixel stats  $\rightarrow$  semantic; counting/tracking  $\rightarrow$  instance) is itself an interview signal.

**Q11. Why does U-Net have skip connections? Quantify from the listing.**

The encoder buys context by downsampling — after a few pools, features know WHAT but only coarsely WHERE; upsampling from the bottleneck alone reconstructs blobs with smeared boundaries. Skips copy each encoder resolution's features to the same-resolution decoder stage: the decoder gets semantics from below and precise localization from the side. Listing 5's ablation on noisy disks: Dice 0.983 -> 0.993, and boundary-pixel accuracy 0.941 -> 0.976 — 2.5x fewer errors exactly where downsampling destroys information. Distinguish from ResNet skips (Chapter 14): those are ADDITIVE within one resolution for gradient flow; U-Net's are CONCATENATED across the encoder-decoder gap for information recovery.

**Q12. Mask R-CNN predicts masks per class with sigmoids, not one softmax over classes. Why? And why RoIAlign?**

Per-class binary masks with sigmoid decouple shape from classification: the box head decides the class; the mask head only answers "which pixels of THIS RoI belong to a *dog*-shaped thing" without classes competing per pixel — softmax coupling was measurably worse in the paper's ablation. RoIAlign: RoI pooling rounds region coordinates to the feature grid (twice — region edges and bin edges); a half-cell rounding at stride 16 is an 8-pixel error in image space, invisible to classification, catastrophic for mask edges. RoIAlign samples features at exact fractional coordinates by bilinear interpolation — no rounding anywhere — and lifted mask AP substantially.

**Q13. Describe the ViT pipeline end to end. What is the ONLY vision-specific part?**

Split the image into  $P \times P$  patches; flatten each to a  $P^2 \cdot C$  vector; ONE learned linear projection to  $d$  — that projection is the entire visual front end, everything after is Chapter 16 verbatim: prepend a learnable CLS token, add positional embeddings, run pre-norm encoder blocks (bidirectional — no causal mask, images have no generation order), classify from the CLS output (or mean-pool the tokens; both work). Listing 6: 8x8 digits, 2x2 patches = 16 tokens, 2 blocks, 4 heads, full backward gradient-checked to  $1.6e-10$ , 92.4% in 3 epochs. The follow-up worth preempting: attention is quadratic in token count, so resolution scales via larger patches, windowed attention (Swin), or pooling hierarchies.

**Q14. Why do CNNs beat ViTs on small datasets, and what reverses it at scale?**

CNNs bake in locality, translation equivariance, and weight sharing — assumptions true of images, so they need not be learned from data. ViT's attention starts with no such priors: any patch may attend anything, and with little data that freedom is variance, not capacity. Listing 6's miniature: ViT 92.4% loses to Chapter 12's plain MLP at 96.0% on 1,400 digits. At scale (ImageNet-21k, JFT-300M) the ranking flips: attention LEARNS locality where useful and long-range structure CNNs cannot express, and ViTs win with better scaling curves. Mitigations for the small-data regime: heavy augmentation + distillation from a CNN teacher (DeiT), hybrid conv stems, or windowed attention (Swin) that restores the locality prior architecturally.

**Q15. Explain CLIP's training objective precisely.**

Batch of  $B$  image-caption pairs; embed via separate towers; L2-normalize both to the unit sphere; similarity matrix  $S = s \cdot I \cdot T^T$  with  $s$  a LEARNABLE temperature (as exp of a log-scale parameter). Loss = symmetric cross-entropy: softmax each row (image  $\rightarrow$  which caption?) and each column (caption  $\rightarrow$  which image?) against the diagonal, averaged. Listing 7 implements it with the normalization backward (project the gradient onto the tangent space:  $dv = (d - v \cdot (d \cdot v)) / ||v_0||$ ) and Adam-free SGD to retrieval accuracy 0.937. Zero-shot classification = embed class-name prompts, nearest caption wins — classification became retrieval, no task head, no fine-tuning.

**Q16. Why does CLIP need a learnable temperature, and what did the listing measure?**

Embeddings live on the unit sphere: cosine similarities are bounded in  $[-1,1]$ , so with scale 1 the softmax over  $B$  candidates is nearly uniform — max/min logit gap 2 — and gradients barely distinguish match from non-match (the same softmax-temperature physics as  $\sqrt{d}$  in Chapter 16 and the sharpening in Chapter 15, third appearance). A learned scale lets the model turn up contrast as embeddings improve. Listing 7: fixed scale 1.0  $\rightarrow$  0.904; learnable  $\rightarrow$  0.937, converging to  $\sim 17$  on its own (real CLIP converges near 100, at clamp). Also worth naming: large batches matter because  $B-1$  in-batch negatives define the task's difficulty — and duplicate-class pairs in a batch are FALSE negatives, absorbed only by batch diversity.

**Q17. Your detector's mAP is fine but users complain about boxes flickering between frames in video. Diagnose.**

mAP is threshold-free and per-frame: it validates the ranking, not score CALIBRATION or temporal stability. Likely causes: scores hovering around the product's confidence threshold (small per-frame feature changes flip detections in and out), NMS instability between two near-equal overlapping boxes, and no temporal smoothing. Fixes: calibrate scores (Chapter 10) and set the operating threshold from a target precision on a validation set rather than 0.5 by reflex; hysteresis (enter at 0.6, persist at 0.4); track-by-detection (Kalman/IoU association) to carry identities; and evaluate with a video metric (MOTA/IDF1), because the offline metric literally cannot see the failure.

**Q18. When does data augmentation HURT? Give measured evidence.**

When the transform breaks the label — augmentation asserts invariance, and false assertions are paid for. Listing 8, 300 training digits: horizontal flip (true invariance for photos, false for digits/text) costs clean accuracy (0.962 -> 0.924 mixed 50/50) and buys zero robustness; always-flipped training collapsed to 0.385 in the unmixed variant. Meanwhile the unaugmented model, 0.962 clean, scores 0.343 under  $\pm 1$ px test jitter — augmentation was NEEDED, just the right one: zero-pad shifts mixed with originals give 0.872 jittered at a 1-point clean cost. Domain list: never flip OCR/text, mind chirality (medical laterality, traffic signs), wrap-around shifts corrupt (`np.roll`, Chapter 13), and aggressive Mixup can erase fine-grained distinctions.

**Q19. What is a feature pyramid network and why do detectors need one?**

Backbone stages trade resolution for semantics: early layers localize precisely but can't tell dog from sofa; late layers know the class but at  $1/32$  resolution a 20px object is sub-pixel. FPN adds a top-down path: upsample deep semantics and ADD laterally-projected earlier features at each scale, producing a pyramid where every level is both semantic and appropriately resolved. Detection heads then run per level, with objects assigned to levels by size (small -> high-res). It is U-Net's idea (Q11) with additive lateral connections in a detector; nearly every modern detector (RetinaNet, Faster R-CNN variants, YOLOs as PAN/BiFPN) carries some descendant of it.

**Q20. Compute the IoU cost of a 2-pixel localization error on a 10x10 box vs a 40x40 box.**

Shift a  $W \times W$  box by 2px in one axis: intersection  $W \cdot (W-2)$ , union  $W \cdot (W+2)$ ,  $\text{IoU} = (W-2)/(W+2)$ .  $W=10$ :  $8/12 = 0.667$  — already below a 0.75 threshold.  $W=40$ :  $38/42 = 0.905$  — comfortably above. Same pixel error, wildly different metric cost: small objects are brutally sensitive to localization noise, which is why COCO reports AP\_small/medium/large separately, why small-object AP lags 20+ points in most detectors, and why high-resolution features (FPN levels, RoIAlign's sub-pixel sampling) matter most for small things. Doing this arithmetic live is a cheap way to show fluency.

**Q21. Dice loss vs pixel cross-entropy for segmentation: when and why?**

Cross-entropy sums per-pixel losses — with 1% foreground, 99% of the gradient budget goes to easy background, and predicting all-background is nearly optimal. Dice loss optimizes  $2 \cdot |P \cap G| / (|P| + |G|)$  directly (soft version with probabilities): scale-free in class frequency, so a tiny lesion's overlap counts as much as a large organ's. Costs: gradients unstable when the structure is absent or tiny (empty-GT images need handling), and it optimizes overlap, not calibration. Standard practice: CE (or focal) + Dice combined; evaluate with Dice/IoU per class plus boundary metrics (Listing 5 reports Dice AND boundary-pixel accuracy for exactly this reason — overlap can look fine while edges are mush).

**Q22. How would you evaluate and deploy zero-shot CLIP classification for a client's product catalog?**

Prompt engineering first: class names alone underperform; template ensembles ("a photo of a {} ", domain phrasings) average into better text anchors — measure, don't assume. Evaluate on a labeled slice: zero-shot accuracy, per-class confusion (CLIP inherits web-data biases and fails on fine-grained or counting tasks), and calibration if thresholds gate actions. If gaps: linear probe on frozen embeddings (cheap, often +10pts over zero-shot with a few hundred labels) before full fine-tuning (which can destroy zero-shot generality — catastrophic forgetting). Serving: text embeddings precompute once; images embed per item; classification is a cosine top-k — an ANN index (Chapter 24) makes it a retrieval system that scales to millions of classes.

**Q23. Why do detection models use different IoU thresholds for assigning training positives vs for evaluation?**

They answer different questions. Training assignment (Listing 3:  $\text{pos} > 0.5$ ,  $\text{neg} < 0.3$ , ignore between) decides which anchors get gradient — the band exists because a 0.4-IoU anchor is genuinely ambiguous: calling it either class injects label noise; best-anchor-per-GT guarantees every object gets at least one learner even if all IoUs are poor. Evaluation IoU (0.5, 0.75, 0.50:0.95) is a product decision about what counts as found. Related refinement: Cascade R-CNN trains successive heads at rising IoU thresholds (0.5/0.6/0.7) because a head trained at 0.5 produces boxes that are only 0.5-good — each stage refines the previous stage's output distribution.

**Q24. A segmentation model scores Dice 0.99 in validation and fails in the clinic. List concrete causes.**

(1) Metric myopia: Dice on large organs hides boundary errors (Listing 5: Dice 0.983 alongside 5.9% boundary error) and tiny-structure failures — report per-structure, boundary distances (Hausdorff), not one mean. (2) Distribution shift: different scanner/protocol/site — the jitter lesson (Listing 8) at institutional scale; validate cross-site. (3) Leakage: patient-level splits, not slice-level — adjacent slices of one patient in train and val is Chapter 4's leakage. (4) Label pathology: annotator style learned as truth. (5) Preprocessing drift between research and deployment pipelines (resampling, windowing). The pattern: the offline metric was real but answered a narrower question than the clinic asks.

**Q25. Estimate the token count and attention cost of ViT-B/16 on a 224x224 image, and at 448x448.**

$224/16 = 14$ , so  $14^2 = 196$  patches + CLS = 197 tokens; attention scores  $197^2 \sim 39\text{k}$  per head per layer — trivial. At 448:  $28^2 = 784$  tokens (+1), scores  $784^2 \sim 615\text{k}$  — 16x the attention cost for 4x the pixels (quadratic in token count = quadratic in area at fixed patch size). Also: the positional embedding table was trained for 196 positions — higher resolution needs 2-D interpolation of the table (standard trick, mild accuracy cost until fine-tuned). This arithmetic is why high-res vision transformers go windowed (Swin), hierarchical, or larger-patch, and why detection/segmentation ViT backbones interpolate positions.

**Q26. Design a real-time defect detector for a factory line: camera at 60fps, defects are 10-50px, latency budget 10ms.**

One-stage anchor-based or anchor-free detector (YOLO-family) — two-stage won't hit 10ms; small input crop-tiling if defects are tiny relative to the frame (downscaling a 4k frame to 640 makes a 10px defect  $\sim 1.5$ px: invisible; tile instead). FPN with an extra high-res level for the 10px end (Q20's arithmetic: small objects need resolution more than semantics). Class imbalance: defects are rare — focal loss, hard-negative mining, and oversample defect frames. Augmentation per Listing 8: photometric jitter yes; flips only if defect chirality is irrelevant; never wrap-shifts. Calibrate scores and pick the operating threshold from the line's cost of misses vs false alarms (Chapter 10); track mAP AND the two operating-point rates. Deploy quantized (Chapter 27), measure p99 latency, and add a drift monitor on the input distribution.

**Q27. Contrast U-Net skips, ResNet skips, and FPN lateral connections — same trick or different?**

Same wiring instinct (shortcut around a lossy path), three different jobs. ResNet (Chapter 14): ADDITIVE identity within one resolution; purpose is optimization — gradient flow through depth; information content is unchanged. U-Net: CONCATENATION across the encoder-decoder gap at matched resolution; purpose is information recovery — spatial detail destroyed by downsampling is re-supplied; optimization is incidental. FPN: additive lateral fusion of upsampled deep semantics with shallow high-res features; purpose is multi-scale REPRESENTATION for scale-assigned prediction heads. Interview trap: calling U-Net "ResNet-style skips" — the concatenation vs addition distinction and the across-resolutions vs within-resolution distinction are the answer.

**Q28. Trace this chapter's measured claims in one pass.**

IoU hand cases 1/3 and 1/7; NMS 13 -> 3 (Listing 1). AP with a duplicate and a hallucination priced exactly: 0.886, falling to 0.371/0.200 at IoU 0.75/0.9 (Listing 2). Anchor encode/decode round-trip error 0.0; assignment with an ignore band (Listing 3). YOLO-core grid detector from scratch: recall 0.81, matched IoU 0.68 (Listing 4). U-Net skip ablation: boundary accuracy 0.941 -> 0.976, 2.5x fewer edge errors (Listing 5). ViT gradient-checked 1.6e-10, 92.4% — beaten by an MLP at small data, the inductive-bias lesson (Listing 6). CLIP: learnable temperature +3pts, scale self-tunes to 17, classification as retrieval 0.937 (Listing 7). Augmentation as asserted invariance: jitter collapse 0.962 -> 0.343 unaugmented, 0.872 with true-invariance shifts, flip a false claim that only costs (Listing 8).